

Fairness Aware Micro-Lending Recommendation System

Project Documentation & Feasibility Report

Step 1: Problem Definition

Team Members:

1. Aditya Jaini
2. Atanu Choudhury
3. Prateek Grover
4. Rajesh Mattaparthi

Problem Statement

This project develops an automated, fairness-aware peer to peer (P2P) micro lending framework. The primary question we seek to answer is: Can we optimize the matching of micro-loans to global lenders while guaranteeing that capital allocation does not discriminate against marginalized demographics?

System Architecture: Core Components

- The system splits the matching and compliance process into two distinct functional layers:
- **The Recommendation Engine (Scoring & Ranking):** Calculates a compatibility match score between an individual loan and a lender's profile. It aligns a lender's stated preferences (such as preferred business sectors or target geographic risk) with the loan's features and country indicators, sorting available loans in descending order so that the highest-scoring matches are surfaced at the very top of the lender's view.
 - Evaluation: We evaluate the quality of this sorted list using Normalized Discounted Cumulative Gain (NDCG), a standard metric that measures how effectively the system places the most relevant items at the top of a user's feed.
- **The Agentic Component (Ethical Policy & Filtering):** Acts as an autonomous, rules-based gatekeeper that reviews the recommendations before they are finalized. It applies strict fairness rules to ensure that the selection and funding rates do not disproportionately favor or exclude borrowers based on sensitive attributes (like gender or specific rural regions).
 - Evaluation: It is evaluated purely on its Guardrail Adherence Rate, tracking the percentage of operations that successfully comply with our fairness rules without creating demographic disparities.

Why This Problem Matters

- **Expands Financial Access:** Micro-entrepreneurs in developing nations lack traditional credit histories. Automated matching bridges this gap to unlock capital.
- **Prevents Algorithmic Bias:** Standard ranking algorithms repeat historical biases against female or rural borrowers. This system uses an active agent to explicitly enforce equity.
- **Scales Lending Workflows:** Micro-finance platforms can process thousands of global applications quickly without human bottlenecks.

Application Feasibility of The Model

Platforms like Kaggle, Kiva's open data snapshots, and the UNData portal archive vital socioeconomic indicators, including regional poverty levels, localized loan transactional structures, and national gender matrices. These historical data channels make it possible to build scalable matching models that can operate programmatically as new loan applications enter the pipeline.

However, practical use still faces several systemic challenges:

- **Scale:** Integrating subnational welfare data with millions of historical individual loan application records requires heavy computational preprocessing and robust pipeline engineering.
- **Consistency:** Discrepancies in regional naming conventions, localized geography data, and varying partner formats across different world regions can yield conflicting or broken data joins.
- **Bias Amplification:** Purely accuracy-driven ranking algorithms naturally favor low-risk, high-volume regions, completely shutting out marginalized borrowers without an active intervention layer.

Success Evaluation Criteria

- **Recommendation Accuracy**
Ranking Optimization: Achieve a Normalized Discounted Cumulative Gain of $NDCG \geq 0.8$ to ensure the highest-matching loans appear at the top of the lender's feed.
- **Practical Utility & Fairness**
Fairness Constraint: Maintain a selection rate variance of $\leq 5\%$ across protected demographic attributes (gender and region). Guardrail Adherence: Achieve a 0% Agentic Guardrail Violation Rate, ensuring the autonomous filter never approves a portfolio allocation that breaches the fairness constraint.

Scope & Constraints

- **Scope:** Focused exclusively on anonymous Kiva crowdfunding records combined with secondary UN economic datasets.
- **Constraints:** Bound entirely to public domain, archived records. Real-time bank telemetry or private personal credit profiles are outside our structural scope.

Step 2: Data Identification and Exploration

Data Source Identification

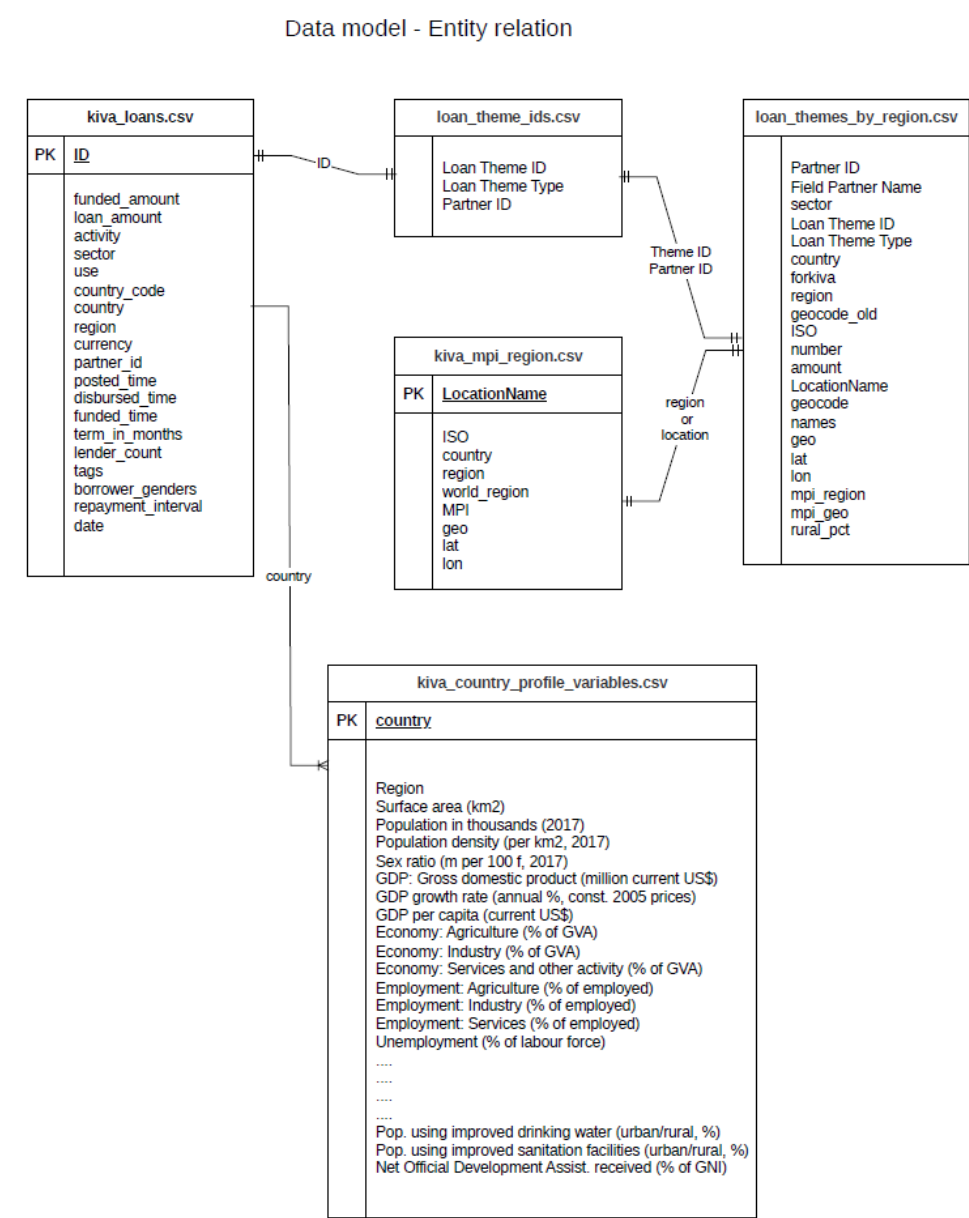
#	Data Source / Variable Role	Source	File Name	Content Details	Granularity	URL
1	Core Loan Transaction Data (Target & Primary Features)	Kaggle - Kiva Crowdfunding	kiva_loans.csv	Loan amounts, specific usage strings, macro business sectors, activity types, repayment structural intervals, and borrower genders.	Individual loan application level	Link
2	Operational Partners & Regional Structural Data (Predictor Variable)	Kaggle - Kiva Crowdfunding	loan_themes_by_region.csv	Identifiers linking Kiva field partner organizations, localized project names, regional operational parameters, latitude/longitude coordinates, and rural population percentage metrics (rural_pct).	Subnational partner regional operation level	Link
3	Subnational Welfare Indices (Predictor Variable)	Kaggle - Kiva Crowdfunding	kiva_mpi_region.csv	Multidimensional Poverty Index (MPI) scores, regional headcount ratios, regional deprivation intensity levels, and broad geographic world sub-regions.	Subnational state/province level across developing countries	Link
4	Partner Structural Mapping (Predictor Variable)	Kaggle - Kiva Crowdfunding	loan_theme_ids.csv	General cross-reference table linking unique loan IDs to specific loan themes, partner IDs, and regional operational buckets.	Unique loan ID level	Link
5	Macroeconomic & Demographic Matrix (Ethical Guardrail Variable)	Kaggle - Country Statistics - UNData	kiva_country_profile_variables.csv	National socio-economic baselines including national sex ratios, gross domestic product (GDP) vectors, annual economic growth percentages, and population densities pre-mapped to Kiva’s distinct country naming conventions.	Country level	Link

Data Availability and Ethical Considerations

All records are fully anonymized, public, and open for academic research. The data contains no personally identifiable information (PII), eliminating privacy risks. The final workflow and pipeline files will be hosted in a public repository for transparent auditing.

Data Exploration and Quality Assessment

We conducted the data exploration and identified the key attributes and features in each of the dataset mentioned above, here’s the data model



A detailed analysis is conducted using Jupyter notebook, it's lengthy hence attached in the last section of this document "[Data Exploration in JupyterLab](#)"

Data Suitability and Project Feasibility

The selected datasets provide excellent, high-density coverage of global P2P micro-lending transactions paired with deep subnational, national economic contexts. Subnational granularity matches our analytical needs, and transactional records provide clear, quantifiable success metrics. The availability of nested indicators ranging from individual borrower genders up to national sex ratios and regional poverty indices matches our analytical need to build a system that balances personalized match accuracy with explicit demographic fairness constraints.

Limitations and Mitigation Strategies

- **Inconsistent Country Formats:** Different data sources spell or format country and region strings differently. Mitigation: We map all records to standardized ISO 3166 alpha-3 country codes.
- **Demographic Skewing:** High-volume regions dominate the raw data, which can bias the recommendation engine. Mitigation: We program explicit demographic fairness rules directly into the agent's policy filtering layer.
- **Sparsity in Subnational Metadata:** Some regional identifiers may lack matching geographical coordinates or MPI index pairings. Mitigation: We will implement proxy imputation strategies using national economic means or leverage the broader partner structural mappings to resolve missing metrics.

Feasibility of Approach

All datasets are publicly accessible and sufficiently detailed for machine learning. The dataset scale is appropriate for standard regression, collaborative filtering, or gradient-boosted tree-based ranking techniques, which are explainable and well-suited to this problem. Established indicators provide meaningful predictors, ensuring both technical feasibility and actionable results for informing regional policy. The entire pipeline runs reliably on standard Python workflows utilizing standard libraries.

Proposed Task Assignments

To meet the project's requirements for exploring alternative approaches, our team of four will be split into two sub-groups. Each group will be responsible for developing one of the two competing proposals for evaluation and modeling.

Sub-Group A:

- Evaluation Proposal #1 Lead (Step 3): Rajesh Mattaparthi

- Modeling Proposal #1 Lead (Step 4): Atanu Choudhury
- Implementation & Testing Task #1 Lead (Step 5): Rajesh Mattaparthi & Atanu Choudhury

Sub-Group B:

- Evaluation Proposal #2 Lead (Step 3): Aditya Jaini
- Modeling Proposal #2 Lead (Step 4): Prateek Grover
- Implementation & Testing Task #2 Lead (Step 5): Aditya Jaini & Prateek Grover

Step 3 : Formal Plan to Evaluate Model Efficacy

Proposal – Sub-Group A (Rajesh Mattaparthi)

As mentioned in the system architecture and core components earlier, this project contains 2 major components - The Recommendation Engine (Scoring & Ranking) and The Agentic Component (Ethical Policy & Filtering). The evaluation plan is designed to catch the risk in each layer match loans to lenders successfully without adding the discrimination against marginalized borrowers.

Evaluation Metrics and justification:

We learned about few metrics in the course such as MAE, MSE, RMSE, R^2 , let us validate the different approaches that is suitable for this project.

Recommendation Engine Evaluation

The primary evaluation objective of the recommendation engine is to match the lenders with suitable qualified borrowers, produce the ranked list of loans, ensure the most relevant loans appear at the top; Hence the evaluation should not just be limited to the prediction accuracy focusing more on the ranking quality.

We considered several possible evaluation methods for the recommendation engine as described below:

Accuracy:

It measures the correct percentage of predictions given by the model out of all predictions it produced. It's commonly used in classification problems.

Suitability check: Accuracy is not suitable for our project as this is not a classification problem to categorize the “approved or rejected”, instead it generates a ranked list of loans based on their relevance to the lender preferences.

Precision:

It measures how many of the top K predictions are actually relevant for the given problem.

Suitability check: Although Precision is useful for measuring the recommendation quality, it only focuses on the top K predictions and ignores the rest of the results in the ranking list, this will not give complete picture; Hence, we consider precision not suitable for our usecase.

Recall:

It measures how many of the relevant items have been considered in the model top K recommendations.

Suitability check: Recall is not suitable for our project because it measures how many relevant loans retrieved by the model but does not consider the ranking position of those loans. Our recommendation engine aims to display the most relevant loans in the top of the list.

MAP (Mean Average Precision):

MAP measures the overall quality of a ranked list by considering both areas – whether it finds relevant items and also how early they appear in the ranking.

Suitability check: MAP is a strong ranking metric and is well suited for this kind of ranking problem. However, it primarily relies on binary relevance which requires predefined labels. Also, it cannot address the other important project objectives such as fairness evaluation. Therefore, although MAP was considered, it was not selected as the primary evaluation metric.

NDCG:

NDCG means Normalized Discounted Cumulative Gain , it measures the quality of the ranked recommendations list by considering the higher importance to relevant items that appear in the top of the list. A high NDCG score means the more relevant recommendations are likely to be placed on the top of the ranking list.

Suitability check: NDCG is a position aware ranking metric that rewards placing the most relevant loans at the top of the recommendation list. It supports graded relevance, allowing loans with different levels of suitability to be ranked appropriately. Since lenders are most likely to review the first few

recommendations, NDCG closely aligns with the objectives of our recommendation system.

Success Criteria:

In the above possible metric options, we have chosen NDCG with a target of $\text{NDCG}@10 \geq 0.80$ established in Checkpoint #1. $k = 10$ is chosen to approximate a lender's first "page" of the recommendations.

Fairness Evaluation

The primary objective of the fairness evaluation is to make sure the ethical agent optimizes for compliance with fairness constraints which is very important for a decision-making process by the lender.

We considered several possible metrics for the fairness evaluation as described below:

Selection Rate Difference:

The idea of this metric is to measure percentage of applications from each protected group, whether the different demographic groups show similar recommendations or not.

For example – if the recommendation shows 45/100 male and 43/100 female, the selection criteria rate would 45% - male 43% - female which is 2% less than the male.

Demographic Parity:

It measures the recommendation outcomes are distributed similarly across protected groups or not. Our model should predict with fairness at the population level.

Equal Opportunity:

It measures the qualified borrowers are treated equally. It will focus on the fairness among the qualified borrowers.

Guardrail Adherence Rate:

This is important metric as it evaluates whether the ethical policy engine is following the predefined fairness rules throughout this process or not. It will help ensure compliance with fairness policies and identify unfair recommendations and evaluate the effectiveness of the ethical agent

From the candidate fairness metrics discussed above, each provides a different perspective for evaluating fairness within recommendation systems. Demographic Parity is effective for identifying population-level bias but assumes similar recommendation rates across all demographic groups without considering other characteristics of borrower. Equal Opportunity focuses on fairness among qualified borrowers; however, it requires clearly defined qualification criteria, which introduces additional assumptions and implementation complexity. While these metrics are valuable for fairness analysis, they are not the primary focus of our evaluation strategy.

Therefore, we selected Selection Rate Difference as the primary fairness metric because it directly measures recommendation disparities across protected demographic groups and aligns with our objective of minimizing demographic bias. In addition, Guardrail Adherence Rate was selected as the secondary metric because it evaluates whether the ethical policy engine consistently enforces predefined fairness rules throughout the recommendation process. Together, these metrics evaluate both the fairness of the recommendation outcomes and the effectiveness of the ethical policy enforcement layer, providing a comprehensive assessment of our fairness-aware recommendation system.

Success Criteria:

We proposed the Selection Rate Variance $\leq 5\%$ and Guardrail Violation Rate = 0% for our model predictions as mentioned in the CheckPoint#1.

Evaluation Data:

As per our initial data analysis (submitted in the CheckPoint#1), we identified “kiva_loans.csv” is the primary data set that contains following attributes; Supporting datasets (loan_themes_by_region, kiva_mpi_region, loan_theme_ids, and country_profile_variables) will also be used during evaluation to reconstruct the same feature set available during training.

- loan amount
- borrower gender
- activity
- sector
- country
- repayment schedule

We consider these are the main features for the recommendation engine model training.

The evaluation dataset will be derived from the same public data sources used in the training; however, we make sure that subsets of records are not being used during model training. We initially thought of using “kiva_country_profile_variables.csv” for the evaluation purpose but it contains different features which could add confusion and lead to poor performance.

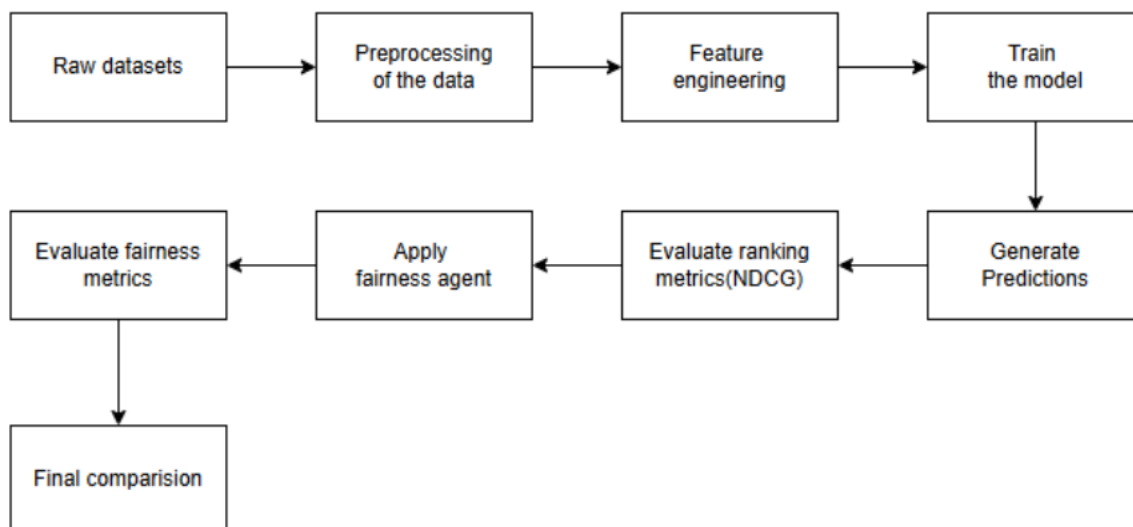
We have considered couple of options of the split for kiva loans.csv dataset :

- » Random split
- » Timestamp split

Though the random split is easy and most commonly used, we planned to go with timestamp based, as the recommendation engine is supposed to recommend the future predictions based on historical data. We will decide the split percentage based on the model training, learning patterns and testing rather than standard split like 80/20 or 70/30.

Evaluation strategy & scalability:

Here is the overview of our evaluation methodology.



We aim to follow the protocols below in this evaluation:

- » Use evaluation datasets that the model has never seen before such as training or validation stages. This will avoid the overfitting issues that usually cause to degrade the model performance after production deployment.
- » Recommendation Engine model performance will be evaluated first and followed by the fairness agent, this will help to improve the quality of recommendations.

- » The final evaluation will assess the complete recommendation pipeline rather than evaluating individual components in isolation, avoiding misleading conclusions that may not be reflecting the overall system performance.
- » Since we need to propose multiple models for this assignment, we want to evaluate them using identical datasets and compare the evaluation metrics. This will give the ability to select the final model that balances the quality recommendations with fairness.
- » As the evaluation requires multiple iterations, we want the evaluation method to be repeatable to produce the consistent results finally. Will document the preprocessing steps and evaluation procedures.

In terms of scalability and production deployment following points will be considered:

- » Since real-world data is much more bigger our final production model should be able to handle the future growth of data sets. Considering this factor, we've chosen the kiva dataset that contains hundreds of thousands of loans and other supplementary data.
- » The other important factor for the production grade solution is the latency & processing times. Following aspects will be taken into account for this final project with reasonable computation limits:
 - How much time the model training took place for the given training data
 - How fast the recommendations should be generated by the model.
- » Will target to minimize the additional processing time for the fairness layer evaluation and ethical validations, guardrail checks will not be significantly delaying the recommendation generation process.
- » We will use the modular architecture that helps identify model drift individually and retraining strategies without rerunning the whole workflow which will save hours of training time and thousands of dollars in computational cost. Also, it will be helpful to the future enhancements.
- » Will add the model performance monitoring to understand the model drifts and take necessary actions to retrain the model well in advance.
- » CI/CD pipelines should be developed using MLOps to minimize the production downtime and human errors .

Proposal – Sub-Group B (Aditya Jaini)

Evaluating a dual-layer recommendation framework comprising a Graph Neural Network (GNN) scoring model and an iterative agentic fairness guardrail requires an evaluation strategy that spans multiple performance dimensions. Standard offline prediction accuracy is insufficient on

its own, the system must also guarantee non-discriminatory capital allocation, execute within tight latency bounds, and scale seamlessly as global loan volume expands. Because the system is split into two distinct functional layers the Recommendation Engine and the Agentic Component the evaluation must assess both raw ranking accuracy and ethical compliance. Furthermore, a third aspect is required to evaluate the system's operational viability.

1. Evaluation Data Strategy

Before evaluating the system, the data pipeline must be established.

- **Data Source:** The evaluation data will be sourced from the exact same origins as the training data: public, anonymized Kiva crowdfunding records and secondary UN economic datasets.
- **Data Split:** The data must be split chronologically into mutually exclusive training, validation, and test sets. It is critical that the evaluation (test) data comes from the same overall distribution to ensure relevant feature mappings (like localized project names, borrower genders, and Multidimensional Poverty Index scores) are consistent. However, the model must never be evaluated on the specific loan records it was trained on, as this would only prove memorization, not predictive capability.

2. Proposal 1: Recommendation Engine Accuracy

This proposal evaluates the raw predictive power of the Graph Neural Network (GNN) and its ability to accurately match lender preferences with loan features before fairness constraints are applied.

- **Identified Metric:** Normalized Discounted Cumulative Gain (NDCG), specifically NDCG@K.
- **Why it is appropriate:** NDCG is a standard metric designed to measure how effectively a system places the most relevant items at the very top of a user's feed. In this context, it ensures the highest-scoring loan matches are properly surfaced to the lender.
- **What constitutes success:** The ranking optimization must achieve an NDCG score of ≥ 0.8 .
- **Evaluation Method:** Run the test set of lenders and candidate loans through the trained GNN. Compute the continuous Match Score for all candidate loans. Sort the loans in descending order based on this score and calculate the NDCG against historical, ground-truth funding data to verify precision.

3. Proposal 2: Ethical Policy and Fairness Compliance

Standard ranking algorithms often repeat historical biases against rural or female borrowers. This proposal evaluates the Agentic Component, which acts as a rules-based gatekeeper to ensure capital allocation does not discriminate against marginalized demographics.

- **Identified Metrics:**
 - Selection Rate Variance.
 - Guardrail Adherence Rate or Agentic Guardrail Violation Rate.
- **Why it is appropriate:** The system explicitly enforces equity to prevent algorithmic bias. These metrics track the percentage of operations that successfully comply with fairness rules without creating demographic disparities.
- **What constitutes success:**
 - The system must maintain a selection rate variance of $\leq 5\%$ across protected demographic attributes, specifically gender and region.
 - The autonomous filter must never approve a portfolio allocation that breaches these fairness constraints, meaning success is defined as a 0% Agentic Guardrail Violation Rate (or a 100% Guardrail Adherence Rate).
- **Evaluation Method:** Following the initial NDCG ranking, feed the candidate slates into the iterative Agentic Guardrail Loop. Monitor the demographic distribution of the final top-K recommendation slate as loans are populated. Audit the final outputs to ensure no demographic quotas were breached during the fallback selection process.

4. Proposal 3: System Scalability and Latency

Simply generating fair, accurate predictions is not enough if the computational overhead is too high. The iterative re-ranking process introduces a loop that evaluates constraints for every top-ranked candidate.

- **Identified Metrics:** Query Latency (measured in milliseconds) and System Throughput (queries per second).
- **Why it is appropriate:** A core goal of this project is to scale lending workflows. Micro-finance platforms must be able to process thousands of global applications quickly without human bottlenecks. If the iterative agentic filter is computationally expensive, it will create system lag and degrade the user experience.
- **What constitutes success:** While specific latency targets are not defined in the source, standard practice dictates that generating a personalized, re-ranked top-K slate should occur in near real-time (e.g., under 500 milliseconds for the 95th percentile of requests) under peak load conditions.
- **Evaluation Method:** Conduct simulated load testing on the architecture. Submit thousands of concurrent lender profiles against the candidate loan database. Measure

the end-to-end time required to run the GNN inference, sort the initial slate, execute the iterative fairness filtering loop, and return the final compliant top-K list.

Identification of Specific Modeling Techniques

To address our core problem—balancing personalized loan-lender match accuracy (Normalized Discounted Cumulative Gain, $NDCG \geq 0.8$) with very strongly defined fairness constraints ($\leq 5\%$ selection rate variance across protected demographic attributes which include gender and region) — we divided ourselves into two sub-groups to evaluate two structural modeling proposals, which preferably will be competing with each other to enable a reliable selection at the end.

Proposal #1: Hybrid Gradient Boosted Ranking and a Heuristic Constrained Post-Filter

- **Modeling Lead:** Atanu Choudhury

1. Technical Framework & System Architecture

- **Layer 1 (Recommendation Engine):** A Gradient Boosted Decision Tree (GBDT) learning-to-rank model using **LightGBM** ([LGBMRanker](#) with [lambdarank objective](#)). The model accepts cross-features between preferences by lender (sector, risk tolerance, etc.) and loan features (loan amount, term, country profile, etc.) to calculate a raw compatibility score $S(u,i)$ for where u indicates the lender and the loan is indicated by i .
- **Layer 2 (Autonomous Agentic Policy Filter):** This layer is a rule-based, deterministic filter engine. The top K recommendations derived from layer 1 computation are passed through a Demographic Parity Policy Filter. If a certain loan batch violates the $\leq 5\%$ selection rate variance rule (e.g., under-representing a certain gender of borrowers or specific rural location tiers), the engine re-ranks the recommendation items by swapping in the eligible loans from underrepresented demographic groups with the highest scores from layer 1.

2. Justification & Suitability

- GBDTs are tailored to process highly complex yet tabular data efficiently and provide feature importance transparently to model-based decision makers. GBDTs are also highly reliable for non-linear data.
- Since we are combining the output from the recommendation engine with the rule-based policy filter, it ensures that the scoring mechanism is free from bias during the process of feature learning.

3. Computational Requirements & Tooling

- **Software Tools:** *lightgbm, scikit-learn, pandas, numpy*.
- **Hardware Needs:** Standard multi-core CPU (8–16 GB RAM); no GPU acceleration required. Due to the nature of the tabular data involved, we are not expecting the processing time to be significant (maximum 10 minutes) for the complete Kiva transaction set.

Step 4: Modeling Strategy & Technical Architecture (Team B)

Proposal #1: Bipartite Graph Neural Network (GraphSAGE) & Agentic Policy Guardrail

- **Modeling Technique:** Bipartite Graph Neural Network (GraphSAGE for link prediction) combined with a post-processing **Agentic Fairness Guardrail**.
- **Why Appropriate:** Micro-lending platforms naturally form complex networks connecting lenders, loans, sectors, and geographic regions. A GNN models these multi-hop structural relationships directly to generate high-dimensional embeddings for initial recommendation scoring.
- **Fairness & Gatekeeper Layer:**
 - Acts as an automated, rule-based policy gatekeeper reviewing top-ranked loan recommendations downstream from the GNN scoring engine.
 - Dynamically re-ranks or swaps in high-scoring loans from underrepresented groups if selection rate variance across protected demographic attributes (gender or subnational region) exceeds the **5% variance threshold**.
 - Enforces a strict **0% Guardrail Violation Rate** to ensure no output breaches ethical compliance rules.
- **Training Strategy:** Trained from scratch on historical loan interaction graph splits.
- **Tools & Libraries:** Python, PyTorch Geometric (torch_geometric / PyG), scikit-learn, and pandas.
- **Computational Needs:** Executes efficiently on a single standard GPU or multi-core CPU architecture.

Proposal #2: Gradient-Boosted Learning-to-Rank (LightGBM) & Explainable Debiasing

- **Modeling Technique:** Gradient-Boosted Decision Trees (LGBMRanker using lambdarank objective) integrated with SHAP (SHapley Additive exPlanations) for interpretability and direct in-model debiasing.
- **Why Appropriate:** Handles structured tabular cross-features (loan terms, field partner history, UN economic indicators) quickly and efficiently.
- **Fairness & Gatekeeper Layer:**

- **In-Model Debiasing:** Adjusts decision tree splits and feature weights during training to prevent sensitive factors (gender and region) from dominating raw suitability scores.
- **Deterministic Post-Filter Engine:** Re-ranks top-K recommendations by swapping in qualified candidate loans from underrepresented demographic groups if selection rate variance exceeds 5%.
- **Explainability:** Leverages SHAP value attribution to provide transparent auditability for why specific loans were recommended or re-ranked.
- **Training Strategy:** Trained from scratch on lender-borrower cross-feature pairs.
- **Tools & Libraries:** Python, lightgbm, shap, scikit-learn, and pandas.
- **Computational Needs:** Consumes less than 10 minutes of execution time on a standard multi-core CPU architecture without requiring GPU acceleration.

Conclusion

In this project, we aim to tackle an issue with a clear social impact: helping improve financial outcomes for underserved entrepreneurs globally. The challenge is well defined, we know how to measure success, and we have the data needed to build a solid machine learning solution. By merging standard P2P lending recommendations with strict agentic policy constraints, we prove that optimization does not have to come at the cost of social fairness.

From our early look at the data, we can see:

- Everything we need is accessible and ethically collected.
- The data quality is sufficient to build reliable predictions.
- Combining different data sources will work smoothly with proper key standardization.
- The size and scope are appropriate for a graduate-level project.

Overall, this project offers a great balance between technical learning and real-world impact, giving us the chance to strengthen our data science skills while contributing to educational equity, demographic fairness, and student/borrower success.

Data Exploration in JupyterLab

Below is the detailed analysis of the data exploration of the datasets mentioned in previous sections.


```
In [57]: # %pip install --upgrade kagglehub pandas
```

```
In [58]: import zipfile
import pandas as pd
import kagglehub
from kagglehub import KaggleDatasetAdapter

# we want to load the kaggle datasets directly from their API, instead of manually
# Hence written this function as it requires more than 1 datasets

def read_datasets_from_kaggle(handle: str, file_name: str):
    if not handle :
        raise ValueError("Dataset handle is mandatory, cannot be empty or None.")
    if not file_name :
        raise ValueError("File name is mandatory, cannot be empty or None.")

    handle = handle.strip()
    file_name = file_name.strip()
    pandas_kwargs = {
        "encoding": "latin-1",
        # "on_bad_lines": "skip",
        "low_memory": False,
    }

    try:
        return kagglehub.dataset_load(
            KaggleDatasetAdapter.PANDAS,
            handle,
            file_name,
            pandas_kwargs=pandas_kwargs,
        )
    except Exception as exc:
        # some of the datasets are named as csv files but are actually ZIP files,
        # so this logic added to handle that case
        temp_file = kagglehub.dataset_download(handle, file_name, force_download=True)

        if zipfile.is_zipfile(temp_file):
            with zipfile.ZipFile(temp_file) as zf:
                names = zf.namelist()
                csv_file_name = next((n for n in names if n.lower().endswith(".csv"))
                                     with zf.open(csv_file_name) as f:
                     return pd.read_csv(f, **pandas_kwargs)

        return pd.read_csv(temp_file, **pandas_kwargs)
```

```
In [59]: # This function is used to explore the dataset and get the basic information about
# such as number of records, columns, data types, nulls, unique values and summary
def explore_my_dataset(df):

    print("_____")
    print("Total records in the dataset :", len(df))
    print("_____")

    # Want to validate the columns and the data types of those columns
```

```

print("Column names and data types:\n", df.dtypes)
print("_____")

# Validate the nulls
print("Nulls in each column:\n", df.isnull().sum())
print("_____")

# Validate the unique records in the given dataset
print("Unique values in each column:\n", df.nunique())
print("_____")

# Summary of the dataset
print("Summary :\n", df.describe(include='all'))
print("_____")

```

```

In [60]: # Pull kiva_loans dataset from Kaggle and load it into a pandas DataFrame
handle = "kiva/data-science-for-good-kiva-crowdfunding"
file_name = "kiva_loans.csv"
df_kiva_loans = read_datasets_from_kaggle(handle, file_name)

#print sample data
print("Sample data from -", file_name)
display(df_kiva_loans)

```

Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.2).
Sample data from - kiva_loans.csv

	id	funded_amount	loan_amount	activity	sector	use	cc
0	653051	300.0	300.0	Fruits & Vegetables	Food	To buy seasonal, fresh fruits to sell.	
1	653053	575.0	575.0	Rickshaw	Transportation	to repair and maintain the auto rickshaw used ...	
2	653068	150.0	150.0	Transportation	Transportation	To repair their old cycle-van and buy another ...	
3	653063	200.0	200.0	Embroidery	Arts	to purchase an embroidery machine and a variet...	
4	653084	400.0	400.0	Milk Sales	Food	to purchase one buffalo.	
...	...	...	...	...	...	...	
671200	1340323	0.0	25.0	Livestock	Agriculture	[True, u'para compara: cemento, arenya y ladri...	
671201	1340316	25.0	25.0	Livestock	Agriculture	[True, u'to start a turducken farm.'] - this l...	
671202	1340334	0.0	25.0	Games	Entertainment	NaN	
671203	1340338	0.0	25.0	Livestock	Agriculture	[True, u'to start a turducken farm.'] - this l...	

	id	funded_amount	loan_amount	activity	sector	use	cc
671204	1340339	0.0	25.0	Livestock	Agriculture	[True, u'to start a turducken farm.'] - this l...	

671205 rows × 20 columns

```
In [61]: #Let's do the data exploration for this dataset

#Basic validation of the dataset
explore_my_dataset(df_kiva_loans)

#additonal validations such as funding, gender categorical data, outliers etc
invalid_amounts = df_kiva_loans[
    (df_kiva_loans["loan_amount"] <= 0) |
    (df_kiva_loans["funded_amount"] < 0) |
    (df_kiva_loans["funded_amount"] > df_kiva_loans["loan_amount"])
]

if invalid_amounts.empty:
    print(" Loan and funded amounts are valid in this dataset")
else:
    print(f" Found {len(invalid_amounts)} invalid amount records.")
    display(invalid_amounts[["id", "loan_amount", "funded_amount"]].head())

# outlier checks
for column in ["loan_amount", "term_in_months", "lender_count"]:
    mean = df_kiva_loans[column].mean()
    std = df_kiva_loans[column].std()

    outliers = df_kiva_loans[
        (df_kiva_loans[column] < mean - 3 * std) |
        (df_kiva_loans[column] > mean + 3 * std)
    ]

    if outliers.empty:
        print(f"PASS: No major outliers in {column}.")
    else:
        print(f"WARNING: {len(outliers)} possible outliers found in {column}.")
```

Total records in the dataset : 671205

Column names and data types:

id	int64
funded_amount	float64
loan_amount	float64
activity	object
sector	object
use	object
country_code	object
country	object
region	object
currency	object
partner_id	float64
posted_time	object
disbursed_time	object
funded_time	object
term_in_months	float64
lender_count	int64
tags	object
borrower_genders	object
repayment_interval	object
date	object

dtype: object

Nulls in each column:

id	0
funded_amount	0
loan_amount	0
activity	0
sector	0
use	4232
country_code	8
country	0
region	56800
currency	0
partner_id	13507
posted_time	0
disbursed_time	2396
funded_time	48331
term_in_months	0
lender_count	0
tags	171416
borrower_genders	4221
repayment_interval	0
date	0

dtype: int64

Unique values in each column:

id	671205
funded_amount	610
loan_amount	479
activity	163
sector	15
use	424912

```

country_code      86
country           87
region           12695
currency          67
partner_id        366
posted_time       667399
disbursed_time    5719
funded_time       498007
term_in_months    148
lender_count      503
tags              86719
borrower_genders  11298
repayment_interval 4
date              1298
dtype: int64

```

Summary :

	id	funded_amount	loan_amount	activity	sector \
count	6.712050e+05	671205.000000	671205.000000	671205	671205
unique	NaN	NaN	NaN	163	15
top	NaN	NaN	NaN	Farming	Agriculture
freq	NaN	NaN	NaN	72955	180302
mean	9.932486e+05	785.995061	842.397107	NaN	NaN
std	1.966113e+05	1130.398941	1198.660073	NaN	NaN
min	6.530470e+05	0.000000	25.000000	NaN	NaN
25%	8.230720e+05	250.000000	275.000000	NaN	NaN
50%	9.927800e+05	450.000000	500.000000	NaN	NaN
75%	1.163653e+06	900.000000	1000.000000	NaN	NaN
max	1.340339e+06	100000.000000	100000.000000	NaN	NaN

	use	country_code \
count	666973	671197
unique	424912	86
top	to buy a water filter to provide safe drinking...	PH
freq	5217	160441
mean	NaN	NaN
std	NaN	NaN
min	NaN	NaN
25%	NaN	NaN
50%	NaN	NaN
75%	NaN	NaN
max	NaN	NaN

	country	region	currency	partner_id \
count	671205	614405	671205	657698.000000
unique	87	12695	67	NaN
top	Philippines	Kaduna	PHP	NaN
freq	160441	10000	160440	NaN
mean	NaN	NaN	NaN	178.199616
std	NaN	NaN	NaN	94.247581
min	NaN	NaN	NaN	9.000000
25%	NaN	NaN	NaN	126.000000
50%	NaN	NaN	NaN	145.000000
75%	NaN	NaN	NaN	204.000000
max	NaN	NaN	NaN	536.000000

	posted_time	disbursed_time \
count	671205	668809
unique	667399	5719
top	2017-05-15 00:00:00+00:00	2017-02-01 08:00:00+00:00
freq	25	2800
mean	NaN	NaN
std	NaN	NaN
min	NaN	NaN
25%	NaN	NaN
50%	NaN	NaN
75%	NaN	NaN
max	NaN	NaN

	funded_time	term_in_months	lender_count \
count	622874	671205.000000	671205.000000
unique	498007	NaN	NaN
top	2016-09-21 13:03:24+00:00	NaN	NaN
freq	33	NaN	NaN
mean	NaN	13.739022	20.590922
std	NaN	8.598919	28.459551
min	NaN	1.000000	0.000000
25%	NaN	8.000000	7.000000
50%	NaN	13.000000	13.000000
75%	NaN	14.000000	24.000000
max	NaN	158.000000	2986.000000

	tags	borrower_genders	repayment_interval	date
count	499789	666984	671205	671205
unique	86719	11298	4	1298
top	user_favorite	female	monthly	2017-03-20
freq	27088	426502	342717	1308
mean	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN

Found 2 invalid amount records.

	id	loan_amount	funded_amount
277188	924149	400.0	425.0
338159	985199	3000.0	3400.0

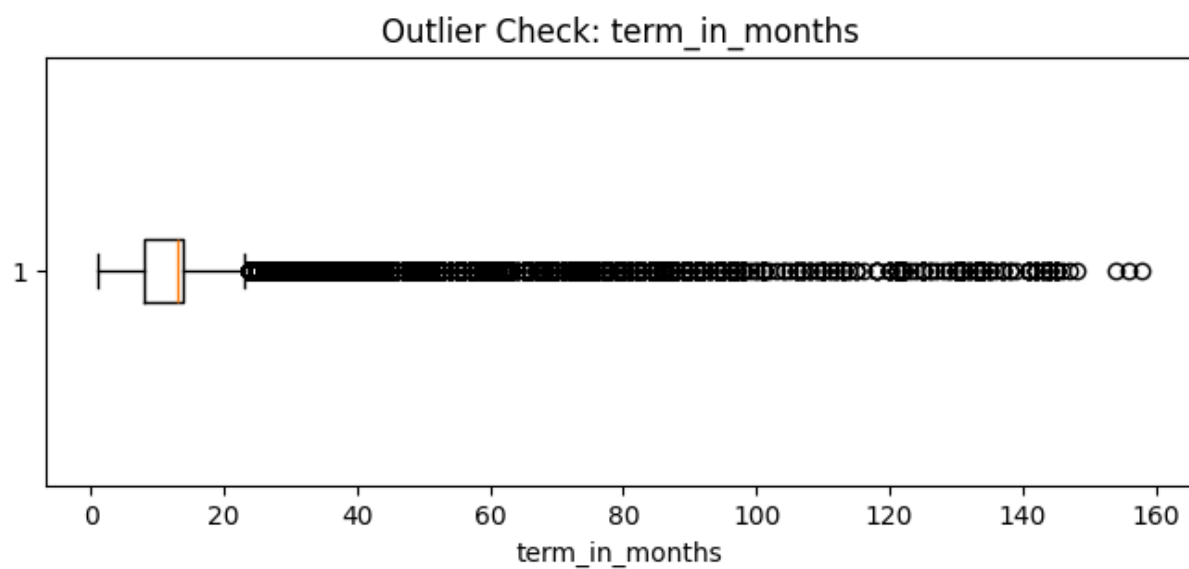
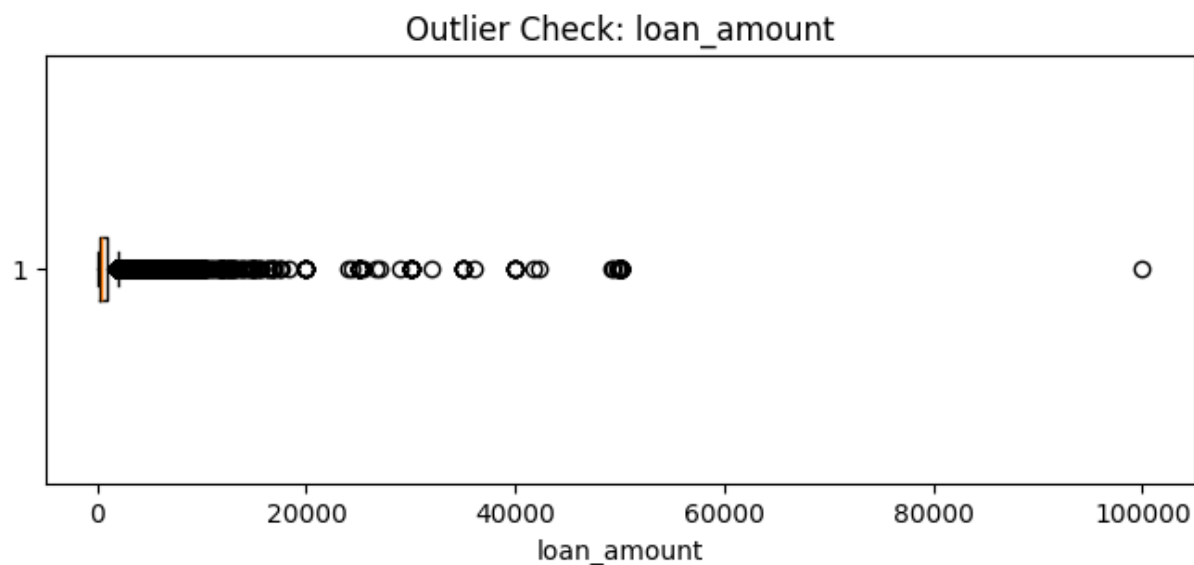
WARNING: 15352 possible outliers found in loan_amount.
 WARNING: 13535 possible outliers found in term_in_months.
 WARNING: 11534 possible outliers found in lender_count.

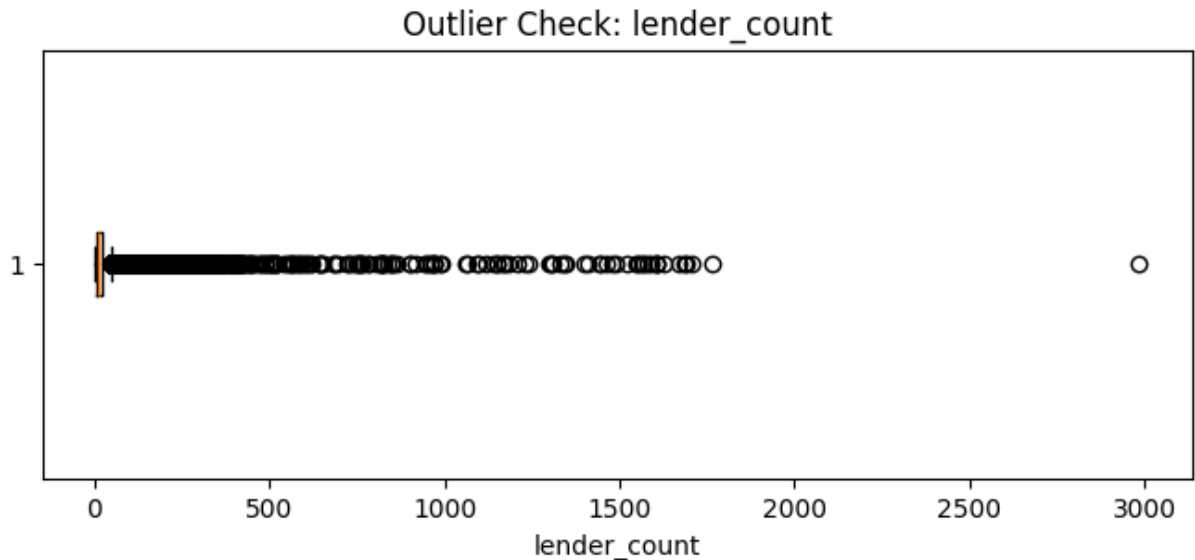
```
In [62]: # want to see the boxplots for the columns to visually check for outliers
import matplotlib.pyplot as plt

columns = ["loan_amount", "term_in_months", "lender_count"]

for column in columns:
```

```
plt.figure(figsize=(8, 3))
plt.boxplot(df_kiva_loans[column].dropna(), vert=False)
plt.title(f"Outlier Check: {column}")
plt.xlabel(column)
plt.show()
```





```
In [63]: # Pull loan_themes_by_region dataset from Kaggle and load it into a pandas DataFrame
handle = "kiva/data-science-for-good-kiva-crowdfunding"
file_name = "loan_themes_by_region.csv"
df_loan_themes_by_region = read_datasets_from_kaggle(handle, file_name)

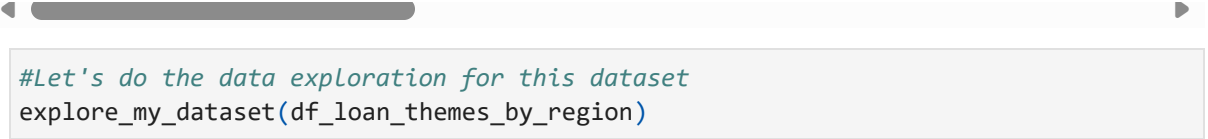
#print sample data
print("Sample data from -", file_name)
display(df_loan_themes_by_region)
```

Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.2).
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.2).
Downloading from https://www.kaggle.com/api/v1/datasets/download/kiva/data-science-for-good-kiva-crowdfunding?dataset_version_number=5&file_name=loan_themes_by_region.csv...

```
100%|████████████████████████████████████████████████████████████████████████████████| 528k/528k [00:00<00:00, 2.14MB/s]
Sample data from - loan_themes_by_region.csv
```

	Partner ID	Field Partner Name	sector	Loan Theme ID	Loan Theme Type	country	forkiva	
0	9	KREDIT Microfinance Institution	General Financial Inclusion	a1050000000slfi	Higher Education	Cambodia	No	I
1	9	KREDIT Microfinance Institution	General Financial Inclusion	a105000000068jPe	Vulnerable Populations	Cambodia	No	Ba
2	9	KREDIT Microfinance Institution	General Financial Inclusion	a1050000000slfi	Higher Education	Cambodia	No	Ba
3	9	KREDIT Microfinance Institution	General Financial Inclusion	a105000000068jPe	Vulnerable Populations	Cambodia	No	
4	9	KREDIT Microfinance Institution	General Financial Inclusion	a1050000002X1Uu	Sanitation	Cambodia	No	
...	...	...	...	...	...	...	...	...
15731	539	N/A, direct to Sevamob	DSE Direct	a10500000081IYI	Health	South Africa	No	
15732	540	N/A, direct to Sanergy	DSE Direct	a10500000081Hlw	Water and Sanitation	Kenya	No	
15733	542	N/A direct to BioLite Inc.	DSE Direct	a1050000008AQrj	Clean Cookstove	Uganda	No	
15734	543	N/A direct to LegWorks Inc.	DSE Direct	a1050000008AQwt	Health	Canada	No	
15735	545	N/A, direct to Solar Home	DSE Direct	a1050000008ASxg	Solar Home Systems	Myanmar (Burma)	No	

15736 rows × 21 columns

A Jupyter Notebook interface showing a code cell. The top bar is light gray with a dark gray progress bar. The code cell has a light gray background and contains two lines of Python code. The first line is a comment in teal, and the second line is a function call in blue.

```
In [64]: #Let's do the data exploration for this dataset  
         explore_my_dataset(df_loan_themes_by_region)
```

Total records in the dataset : 15736

Column names and data types:

Partner ID	int64
Field Partner Name	object
sector	object
Loan Theme ID	object
Loan Theme Type	object
country	object
forkiva	object
region	object
geocode_old	object
ISO	object
number	int64
amount	int64
LocationName	object
geocode	object
names	object
geo	object
lat	float64
lon	float64
mpi_region	object
mpi_geo	object
rural_pct	float64

dtype: object

Nulls in each column:

Partner ID	0
Field Partner Name	0
sector	0
Loan Theme ID	0
Loan Theme Type	0
country	0
forkiva	0
region	0
geocode_old	14536
ISO	14
number	0
amount	0
LocationName	0
geocode	2074
names	2075
geo	0
lat	2074
lon	2074
mpi_region	14
mpi_geo	6065
rural_pct	1392

dtype: int64

Unique values in each column:

Partner ID	302
Field Partner Name	302
sector	11
Loan Theme ID	718

Loan Theme Type	170
country	79
forkiva	2
region	9526
geocode_old	340
ISO	77
number	638
amount	2532
LocationName	9561
geocode	6557
names	6275
geo	6558
lat	6550
lon	6015
mpi_region	392
mpi_geo	335
rural_pct	56
dtype: int64	

Summary :

	Partner ID	Field Partner Name	sector \
count	15736.000000	15736	15736
unique	NaN	302	11
top	NaN	Alalay sa Kaunlaran (ASKI)	General Financial Inclusion
freq	NaN	1207	13679
mean	191.376144	NaN	NaN
std	118.705003	NaN	NaN
min	9.000000	NaN	NaN
25%	123.000000	NaN	NaN
50%	154.000000	NaN	NaN
75%	217.000000	NaN	NaN
max	545.000000	NaN	NaN

	Loan Theme ID	Loan Theme Type	country	forkiva	region \
count	15736	15736	15736	15736	15736
unique	718	170	79	2	9526
top	a1050000000wf0V	General	Philippines	No	Chouf
freq	1152	5661	3467	13211	25
mean	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN

	geocode_old	ISO	...	amount	LocationName \
count	1200	15722	...	1.573600e+04	15736
unique	340	77	...	NaN	9561
top	(-1.2833333, 36.8166667)	PHL	...	NaN	Chouf, Lebanon
freq	20	3467	...	NaN	25
mean	NaN	NaN	...	2.003991e+04	NaN
std	NaN	NaN	...	9.636941e+04	NaN
min	NaN	NaN	...	2.500000e+01	NaN
25%	NaN	NaN	...	9.500000e+02	NaN
50%	NaN	NaN	...	2.600000e+03	NaN

75%	NaN	NaN	...	8.750000e+03	NaN
max	NaN	NaN	...	4.929900e+06	NaN

	geocode	names	geo \
count	13662	13661	15736
unique	6557	6275	6558
top	[(19.7126764, 105.8393447)]	Philippines	(1000.0, 1000.0)
freq	43	137	2074
mean	NaN	NaN	NaN
std	NaN	NaN	NaN
min	NaN	NaN	NaN
25%	NaN	NaN	NaN
50%	NaN	NaN	NaN
75%	NaN	NaN	NaN
max	NaN	NaN	NaN

	lat	lon	mpi_region \
count	13662.000000	13662.000000	15722
unique	NaN	NaN	392
top	NaN	NaN	Northern Mindanao, Philippines
freq	NaN	NaN	1174
mean	14.328878	29.433569	NaN
std	16.757689	83.255739	NaN
min	-34.610548	-172.790661	NaN
25%	6.266728	-71.967463	NaN
50%	13.484101	41.899993	NaN
75%	22.189940	106.677321	NaN
max	51.314017	159.972900	NaN

	mpi_geo	rural_pct
count	9671	14344.000000
unique	335	NaN
top	(8.020163499999999, 124.6856509)	NaN
freq	1174	NaN
mean	NaN	68.084635
std	NaN	26.550064
min	NaN	0.000000
25%	NaN	60.000000
50%	NaN	73.000000
75%	NaN	85.000000
max	NaN	100.000000

[11 rows x 21 columns]

```
In [65]: # Pull kiva_mpi_region_locations.csv dataset from Kaggle and load it into a pandas
handle = "kiva/data-science-for-good-kiva-crowdfunding"
file_name = "kiva_mpi_region_locations.csv"
df_kiva_mpi_region = read_datasets_from_kaggle(handle, file_name)

#print sample data
print("Sample data from -", file_name)
display(df_kiva_mpi_region)
```

Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.2).
Sample data from - kiva_mpi_region_locations.csv

	LocationName	ISO	country	region	world_region	MPI	geo
0	Badakhshan, Afghanistan	AFG	Afghanistan	Badakhshan	South Asia	0.387	(36.7347725, 70.81199529999999)
1	Badghis, Afghanistan	AFG	Afghanistan	Badghis	South Asia	0.466	(35.1671339, 63.7695384)
2	Baghlan, Afghanistan	AFG	Afghanistan	Baghlan	South Asia	0.300	(35.8042947, 69.2877535)
3	Balkh, Afghanistan	AFG	Afghanistan	Balkh	South Asia	0.301	(36.7550603, 66.8975372)
4	Bamyan, Afghanistan	AFG	Afghanistan	Bamyan	South Asia	0.325	(34.8100067, 67.8212104)
...	...	...	...	...	...	...	...
2767	NaN	NaN	NaN	NaN	NaN	NaN	(1000.0, 1000.0)
2768	NaN	NaN	NaN	NaN	NaN	NaN	(1000.0, 1000.0)
2769	NaN	NaN	NaN	NaN	NaN	NaN	(1000.0, 1000.0)
2770	NaN	NaN	NaN	NaN	NaN	NaN	(1000.0, 1000.0)
2771	NaN	NaN	NaN	NaN	NaN	NaN	(1000.0, 1000.0)

2772 rows × 9 columns



```
In [66]: #Let's do the data exploration for this dataset
         explore_my_dataset(df_kiva_mpi_region)
```

Total records in the dataset : 2772

Column names and data types:

LocationName	object
ISO	object
country	object
region	object
world_region	object
MPI	float64
geo	object
lat	float64
lon	float64

dtype: object

Nulls in each column:

LocationName	1788
ISO	1764
country	1764
region	1788
world_region	1764
MPI	1788
geo	0
lat	1880
lon	1880

dtype: int64

Unique values in each column:

LocationName	984
ISO	102
country	102
region	928
world_region	6
MPI	452
geo	881
lat	880
lon	787

dtype: int64

Summary :

	LocationName	ISO	country	region	world_region \
count	984	1008	1008	984	1008
unique	984	102	102	928	6
top	Midlands, Zimbabwe	NGA	Nigeria	Eastern	Sub-Saharan Africa
freq	1	37	37	8	432
mean	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN

	MPI	geo	lat	lon
count	984.000000	2772	892.000000	892.000000
unique	NaN	881	NaN	NaN

top	NaN	(1000.0, 1000.0)	NaN	NaN
freq	NaN	1880	NaN	NaN
mean	0.211330	NaN	9.169710	16.635888
std	0.183621	NaN	16.484531	61.234566
min	0.000000	NaN	-34.947896	-122.747131
25%	0.053000	NaN	-1.027901	-12.819854
50%	0.155000	NaN	11.211379	26.419389
75%	0.341500	NaN	18.084292	47.019436
max	0.744000	NaN	49.264748	138.581284

```
In [67]: # Pull loan_theme_ids dataset from Kaggle and load it into a pandas DataFrame
handle = "kiva/data-science-for-good-kiva-crowdfunding"
file_name = "loan_theme_ids.csv"
df_loan_theme_ids = read_datasets_from_kaggle(handle, file_name)

#print sample data
print("Sample data from -", file_name)
display(df_loan_theme_ids)
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.1
3), please consider upgrading to the latest version (1.0.2).
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.1
3), please consider upgrading to the latest version (1.0.2).
Downloading from https://www.kaggle.com/api/v1/datasets/download/kiva/data-science-f
or-good-kiva-crowdfunding?dataset_version_number=5&file_name=loan_theme_ids.csv...
```

Sample data from - loan_theme_ids.csv

	id	Loan Theme ID	Loan Theme Type	Partner ID
0	638631	a1050000000skGI	General	151.0
1	640322	a1050000000skGI	General	151.0
2	641006	a1050000002X1ij	Higher Education	160.0
3	641019	a1050000002X1ij	Higher Education	160.0
4	641594	a1050000002VbsW	Subsistence Agriculture	336.0
...	...	...	...	...
779087	1444237	a1050000000wf0h	General	136.0
779088	1444238	a1050000000wf0h	General	136.0
779089	1444240	a1050000000wf0h	General	136.0
779090	1444241	a1050000000wf22	General	245.0
779091	1444243	a1050000000wf22	General	245.0

779092 rows x 4 columns

```
In [68]: #Let's do the data exploration for this dataset
         explore_my_dataset(df_loan_theme_ids)
```

Total records in the dataset : 779092

Column names and data types:

id	int64
Loan Theme ID	object
Loan Theme Type	object
Partner ID	float64

dtype: object

Nulls in each column:

id	0
Loan Theme ID	14813
Loan Theme Type	14813
Partner ID	14813

dtype: int64

Unique values in each column:

id	779092
Loan Theme ID	956
Loan Theme Type	203
Partner ID	384

dtype: int64

Summary :

	id	Loan Theme ID	Loan Theme Type	Partner ID
count	7.790920e+05	764279	764279	764279.000000
unique	NaN	956	203	NaN
top	NaN	a1050000000wf0q	General	NaN
freq	NaN	110264	380693	NaN
mean	1.047475e+06	NaN	NaN	180.825840
std	2.282538e+05	NaN	NaN	97.914029
min	6.386310e+05	NaN	NaN	9.000000
25%	8.499768e+05	NaN	NaN	126.000000
50%	1.046528e+06	NaN	NaN	145.000000
75%	1.244768e+06	NaN	NaN	204.000000
max	1.444243e+06	NaN	NaN	557.000000

```
In [69]: # Pull kiva_country_profile_variables dataset from Kaggle and load it into a pandas
handle = "sudalairajkumar/undata-country-profiles"
file_name = "kiva_country_profile_variables.csv"
df_kiva_country_profile_variables = read_datasets_from_kaggle(handle, file_name)

#print sample data
print("Sample data from -", file_name)
display(df_kiva_country_profile_variables)
```

Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.2).

Sample data from - kiva_country_profile_variables.csv

	country	Region	Surface area (km2)	Population in thousands (2017)	Population density (per km2, 2017)	Sex ratio (m per 100 f, 2017)	GDP: Gross domestic product (million current US\$)	GDP growth rate (annual %, const. 2005 prices)
0	Afghanistan	SouthernAsia	652864	35530	54.4	106.3	20270	-2.4
1	Albania	SouthernEurope	28748	2930	106.9	101.9	11541	2.6
2	Armenia	WesternAsia	29743	2930	102.9	88.8	10529	3.0
3	Azerbaijan	WesternAsia	86600	9828	118.9	99.3	53049	0.7
4	Belize	CentralAmerica	22966	375	16.4	99.2	1721	1.2
...	...	...	...	...	...	...	...	...
81	Vietnam	South-easternAsia	330967	95541	308.1	98.0	193241	6.7
82	Virgin Islands	Caribbean	347	105	299.7	91.1	-99	-99
83	Yemen	WesternAsia	527968	28250	53.5	102.1	29688	-28.1
84	Zambia	EasternAfrica	752612	17094	23.0	98.5	21255	2.9
85	Zimbabwe	EasternAfrica	390757	16530	42.7	95.0	13893	1.1

86 rows × 50 columns



```
In [70]: #Let's do the data exploration for this dataset
         explore_my_dataset(df_kiva_country_profile_variables)
```

Total records in the dataset : 86

Column names and data types:

country	object
Region	object
Surface area (km2)	int64
Population in thousands (2017)	int64
Population density (per km2, 2017)	float64
Sex ratio (m per 100 f, 2017)	float64
GDP: Gross domestic product (million current US\$)	int64
GDP growth rate (annual %, const. 2005 prices)	object
GDP per capita (current US\$)	float64
Economy: Agriculture (% of GVA)	float64
Economy: Industry (% of GVA)	float64
Economy: Services and other activity (% of GVA)	float64
Employment: Agriculture (% of employed)	float64
Employment: Industry (% of employed)	float64
Employment: Services (% of employed)	float64
Unemployment (% of labour force)	object
Labour force participation (female/male pop. %)	object
Agricultural production index (2004-2006=100)	int64
Food production index (2004-2006=100)	int64
International trade: Exports (million US\$)	int64
International trade: Imports (million US\$)	int64
International trade: Balance (million US\$)	int64
Balance of payments, current account (million US\$)	object
Population growth rate (average annual %)	object
Urban population (% of total population)	float64
Urban population growth rate (average annual %)	float64
Fertility rate, total (live births per woman)	float64
Life expectancy at birth (females/males, years)	object
Population age distribution (0-14 / 60+ years, %)	object
International migrant stock (000/% of total pop.)	object
Refugees and others of concern to UNHCR (in thousands)	object
Infant mortality rate (per 1000 live births)	float64
Health: Total expenditure (% of GDP)	float64
Health: Physicians (per 1000 pop.)	object
Education: Government expenditure (% of GDP)	object
Education: Primary gross enrol. ratio (f/m per 100 pop.)	object
Education: Secondary gross enrol. ratio (f/m per 100 pop.)	object
Education: Tertiary gross enrol. ratio (f/m per 100 pop.)	object
Seats held by women in national parliaments %	float64
Mobile-cellular subscriptions (per 100 inhabitants)	object
Mobile-cellular subscriptions (per 100 inhabitants).1	float64
Individuals using the Internet (per 100 inhabitants)	int64
Threatened species (number)	float64
Forested area (% of land area)	object
CO2 emission estimates (million tons/tons per capita)	int64
Energy production, primary (Petajoules)	int64
Energy supply per capita (Gigajoules)	object
Pop. using improved drinking water (urban/rural, %)	object
Pop. using improved sanitation facilities (urban/rural, %)	float64
Net Official Development Assist. received (% of GNI)	int64
dtype:	object

Nulls in each column:	
country	0
Region	0
Surface area (km2)	0
Population in thousands (2017)	0
Population density (per km2, 2017)	0
Sex ratio (m per 100 f, 2017)	0
GDP: Gross domestic product (million current US\$)	0
GDP growth rate (annual %, const. 2005 prices)	0
GDP per capita (current US\$)	0
Economy: Agriculture (% of GVA)	0
Economy: Industry (% of GVA)	0
Economy: Services and other activity (% of GVA)	0
Employment: Agriculture (% of employed)	0
Employment: Industry (% of employed)	0
Employment: Services (% of employed)	0
Unemployment (% of labour force)	0
Labour force participation (female/male pop. %)	0
Agricultural production index (2004-2006=100)	0
Food production index (2004-2006=100)	0
International trade: Exports (million US\$)	0
International trade: Imports (million US\$)	0
International trade: Balance (million US\$)	0
Balance of payments, current account (million US\$)	0
Population growth rate (average annual %)	0
Urban population (% of total population)	0
Urban population growth rate (average annual %)	0
Fertility rate, total (live births per woman)	0
Life expectancy at birth (females/males, years)	0
Population age distribution (0-14 / 60+ years, %)	0
International migrant stock (000/% of total pop.)	0
Refugees and others of concern to UNHCR (in thousands)	0
Infant mortality rate (per 1000 live births)	0
Health: Total expenditure (% of GDP)	0
Health: Physicians (per 1000 pop.)	0
Education: Government expenditure (% of GDP)	0
Education: Primary gross enrol. ratio (f/m per 100 pop.)	0
Education: Secondary gross enrol. ratio (f/m per 100 pop.)	0
Education: Tertiary gross enrol. ratio (f/m per 100 pop.)	0
Seats held by women in national parliaments %	0
Mobile-cellular subscriptions (per 100 inhabitants)	0
Mobile-cellular subscriptions (per 100 inhabitants).1	0
Individuals using the Internet (per 100 inhabitants)	0
Threatened species (number)	0
Forested area (% of land area)	0
CO2 emission estimates (million tons/tons per capita)	0
Energy production, primary (Petajoules)	0
Energy supply per capita (Gigajoules)	0
Pop. using improved drinking water (urban/rural, %)	0
Pop. using improved sanitation facilities (urban/rural, %)	0
Net Official Development Assist. received (% of GNI)	0
dtype: int64	

Unique values in each column:	
country	86
Region	19

Surface area (km2)	86
Population in thousands (2017)	85
Population density (per km2, 2017)	86
Sex ratio (m per 100 f, 2017)	64
GDP: Gross domestic product (million current US\$)	85
GDP growth rate (annual %, const. 2005 prices)	57
GDP per capita (current US\$)	84
Economy: Agriculture (% of GVA)	74
Economy: Industry (% of GVA)	73
Economy: Services and other activity (% of GVA)	75
Employment: Agriculture (% of employed)	82
Employment: Industry (% of employed)	75
Employment: Services (% of employed)	79
Unemployment (% of labour force)	66
Labour force participation (female/male pop. %)	86
Agricultural production index (2004-2006=100)	57
Food production index (2004-2006=100)	55
International trade: Exports (million US\$)	84
International trade: Imports (million US\$)	84
International trade: Balance (million US\$)	84
Balance of payments, current account (million US\$)	83
Population growth rate (average annual %)	36
Urban population (% of total population)	83
Urban population growth rate (average annual %)	44
Fertility rate, total (live births per woman)	43
Life expectancy at birth (females/males, years)	85
Population age distribution (0-14 / 60+ years, %)	85
International migrant stock (000/% of total pop.)	86
Refugees and others of concern to UNHCR (in thousands)	71
Infant mortality rate (per 1000 live births)	78
Health: Total expenditure (% of GDP)	56
Health: Physicians (per 1000 pop.)	28
Education: Government expenditure (% of GDP)	45
Education: Primary gross enrol. ratio (f/m per 100 pop.)	82
Education: Secondary gross enrol. ratio (f/m per 100 pop.)	75
Education: Tertiary gross enrol. ratio (f/m per 100 pop.)	69
Seats held by women in national parliaments %	75
Mobile-cellular subscriptions (per 100 inhabitants)	83
Mobile-cellular subscriptions (per 100 inhabitants).1	81
Individuals using the Internet (per 100 inhabitants)	81
Threatened species (number)	83
Forested area (% of land area)	84
CO2 emission estimates (million tons/tons per capita)	77
Energy production, primary (Petajoules)	47
Energy supply per capita (Gigajoules)	81
Pop. using improved drinking water (urban/rural, %)	85
Pop. using improved sanitation facilities (urban/rural, %)	77
Net Official Development Assist. received (% of GNI)	1
dtype: int64	

Summary :

	country	Region	Surface area (km2) \
count	86	86	8.600000e+01
unique	86	19	NaN
top	Afghanistan	EasternAfrica	NaN
freq	1	12	NaN

mean	NaN	NaN	7.673924e+05
std	NaN	NaN	1.738184e+06
min	NaN	NaN	3.470000e+02
25%	NaN	NaN	4.927825e+04
50%	NaN	NaN	2.400435e+05
75%	NaN	NaN	7.552295e+05
max	NaN	NaN	9.833517e+06

	Population in thousands (2017)	Population density (per km2, 2017)	\
count	8.600000e+01	86.000000	
unique	NaN	NaN	
top	NaN	NaN	
freq	NaN	NaN	
mean	6.590220e+04	131.374419	
std	2.113548e+05	148.505419	
min	1.050000e+02	2.000000	
25%	4.909750e+03	36.350000	
50%	1.239200e+04	79.050000	
75%	3.758875e+04	149.000000	
max	1.409517e+06	817.400000	

	Sex ratio (m per 100 f, 2017)	\
count	86.000000	
unique	NaN	
top	NaN	
freq	NaN	
mean	99.086047	
std	4.182067	
min	86.000000	
25%	96.925000	
50%	99.350000	
75%	101.550000	
max	113.100000	

	GDP: Gross domestic product (million current US\$)	\
count	8.600000e+01	
unique	NaN	
top	NaN	
freq	NaN	
mean	4.740389e+05	
std	2.279963e+06	
min	-9.900000e+01	
25%	8.191000e+03	
50%	2.051150e+04	
75%	8.473700e+04	
max	1.803665e+07	

	GDP growth rate (annual %, const. 2005 prices)	\
count	86	
unique	57	
top	7.0	
freq	4	
mean	NaN	
std	NaN	
min	NaN	
25%	NaN	

50%	NaN
75%	NaN
max	NaN

	GDP per capita (current US\$)	Economy: Agriculture (% of GVA)	...	\
count	86.000000	86.000000	...	
unique	NaN	NaN	...	
top	NaN	NaN	...	
freq	NaN	NaN	...	
mean	4442.337209	14.626744	...	
std	7809.939810	21.916507	...	
min	-99.000000	-99.000000	...	
25%	907.300000	6.975000	...	
50%	2077.400000	14.300000	...	
75%	4760.450000	23.675000	...	
max	56053.800000	70.800000	...	

	Mobile-cellular subscriptions (per 100 inhabitants).1	\
count	86.000000	
unique	NaN	
top	NaN	
freq	NaN	
mean	32.851163	
std	21.392152	
min	1.800000	
25%	17.300000	
50%	25.250000	
75%	50.975000	
max	79.500000	

	Individuals using the Internet (per 100 inhabitants)	\
count	86.000000	
unique	NaN	
top	NaN	
freq	NaN	
mean	317.732558	
std	408.412318	
min	18.000000	
25%	86.250000	
50%	150.000000	
75%	335.250000	
max	2358.000000	

	Threatened species (number)	Forested area (% of land area)	\
count	86.000000	86	
unique	NaN	84	
top	NaN	-99	
freq	NaN	3	
mean	30.533721	NaN	
std	27.139741	NaN	
min	-99.000000	NaN	
25%	10.375000	NaN	
50%	33.050000	NaN	
75%	50.075000	NaN	
max	98.300000	NaN	

CO2 emission estimates (million tons/tons per capita) \	
count	86.000000
unique	NaN
top	NaN
freq	NaN
mean	3660.093023
std	14382.122938
min	-99.000000
25%	57.250000
50%	139.500000
75%	983.750000
max	101394.000000

Energy production, primary (Petajoules) \	
count	86.000000
unique	NaN
top	NaN
freq	NaN
mean	32.290698
std	42.720670
min	-99.000000
25%	14.000000
50%	26.000000
75%	37.500000
max	289.000000

Energy supply per capita (Gigajoules) \	
count	86
unique	81
top	100.0/100.0
freq	6
mean	NaN
std	NaN
min	NaN
25%	NaN
50%	NaN
75%	NaN
max	NaN

Pop. using improved drinking water (urban/rural, %) \	
count	86
unique	85
top	100.0/100.0
freq	2
mean	NaN
std	NaN
min	NaN
25%	NaN
50%	NaN
75%	NaN
max	NaN

Pop. using improved sanitation facilities (urban/rural, %) \	
count	86.000000
unique	NaN
top	NaN

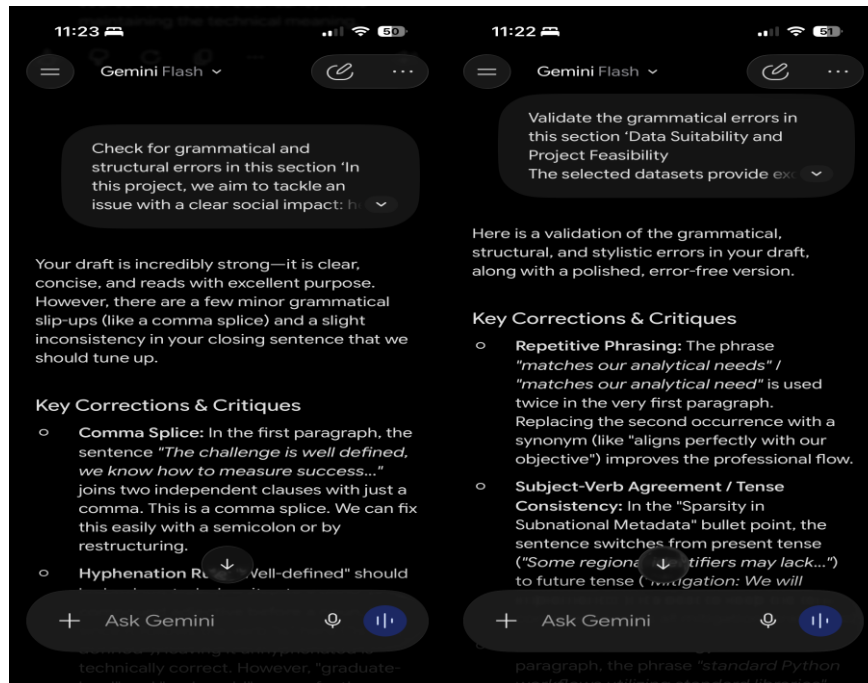
freq	NaN
mean	-0.299767
std	26.027691
min	-99.000000
25%	0.377500
50%	2.910000
75%	6.262500
max	61.730000

Net Official Development Assist. received (% of GNI)	
count	86.0
unique	NaN
top	NaN
freq	NaN
mean	-99.0
std	0.0
min	-99.0
25%	-99.0
50%	-99.0
75%	-99.0
max	-99.0

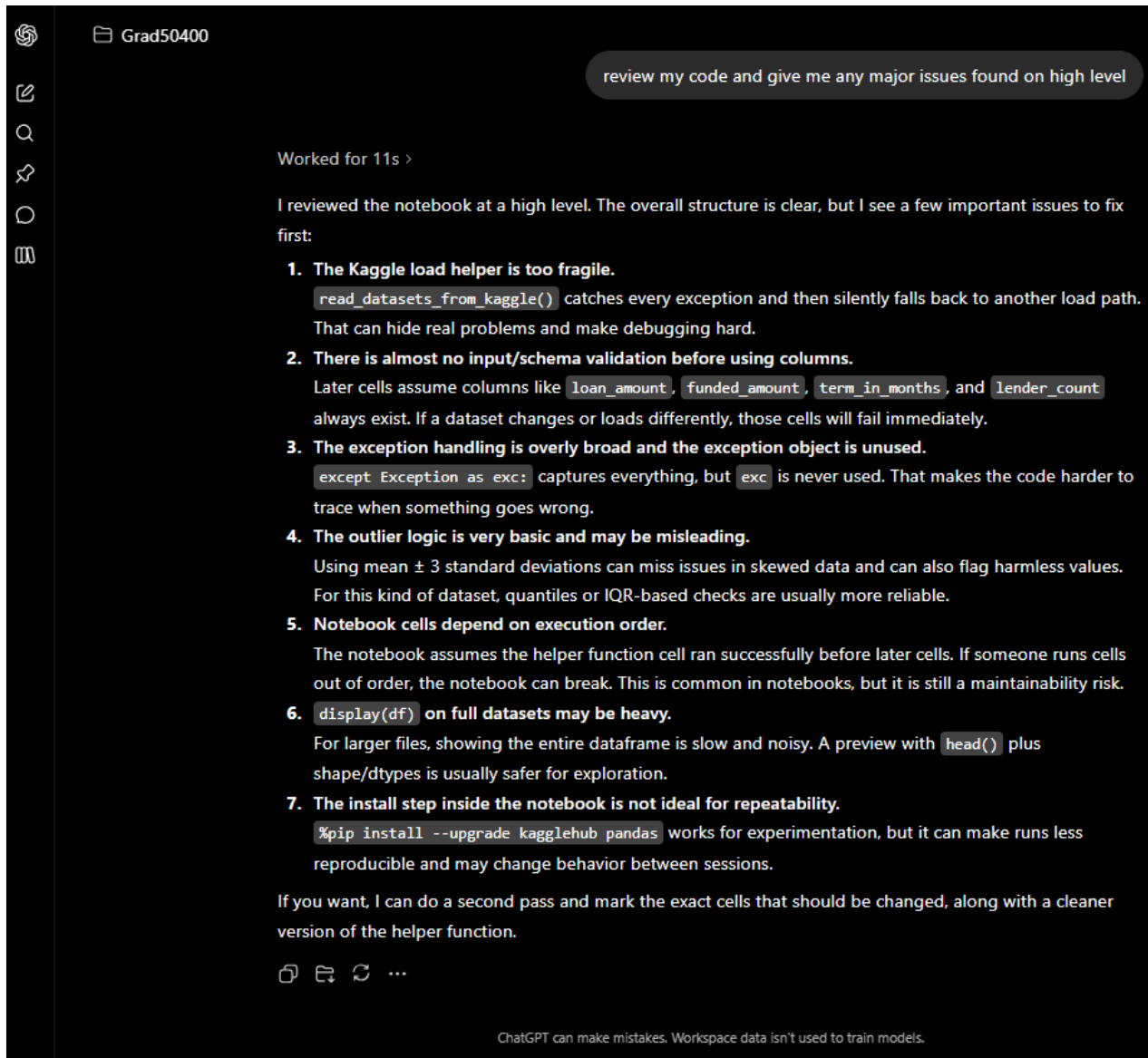
[11 rows x 50 columns]

Appendix - AI Usage

- Gemini – Was used to format the structure of the doc and identify the grammatical and spelling errors in the last stages of formatting the document. Below are the screenshots



Used Chatgpt only to do the final code review for the data exploration, some of them fixed and ignored the rest.



For CheckPoint#2 Step3, I used ChatGPT to validate the spelling and grammar mistakes, find the full chat history below:



ChatGPT - Step3.pdf